Introdução - Computação Gráfica

GBC204 - Computação Gráfica

prof: Daniel Duarte Abdala

Horário das Aulas:

Ter: 08:50h ~ 10:30h
Qui: 08:50h ~ 10:30h

Atendimento:

Ter: 10:30h ~ 11:30h (1B121)
Qui: 10:30h ~ 11:30h (1B121)

Datas das Avaliações:

Avaliação Teórica:
Trabalho (Módulo 1)
Trabalho (Módulo 2)
Trabalho (Módulo 3)
Trabalho (Módulo 4)

Aulas para complementação de carga horária:

2ha (09/05) - Compl. 1 : Assistir Filme e Escrever Resenha
2ha (16/05) - Compl. 2 : Revisar o Algoritmo DDA e Reportar
2ha (23/05) - Compl. 3: Comp. pelo menos 3 modelos de cor
2ha (30/05) - Compl. 4: Modelar um objeto no Blender
2ha (06/06) - Compl. 5: Aplicar uma Textura no objeto modelado
2ha (20/06) - Compl. 6: Gerar uma animação no Blender

Conteúdo Programático

2. **EMENTA**
Introdução à Computação Gráfica; Geração de Primitivas Gráficas; Algoritmos de Recorte; Transformações geométricas; Pipeline de exibição – malhas de polígonos; Processamento de sólidos; Cores; Rendering: Remoção de linhas e superfícies ocultas, Modelos de Iluminação, Modelos de Tonalização, Modelos globais de iluminação.

Aulas

30 aulas presenciais
6 aulas de compl.

MARÇO

D	S	T	Q	Q	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

21 – Término das aulas 2025/2
23 a 26 – Período outras atividades acadêmicas
26 – Término do semestre 2025/2
31 – Prazo para lançamento de notas e faltas 2025/2

01 e 02/04 – Solicitação de matrícula
08 e 09/04 – Ajuste de matrícula
09/04 – Início do semestre 2026-1

ABRIL 3 aulas

D	S	T	Q	Q	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

1 e 2 – Solicitação de matrícula portal – 2026/1
3 – Feriado – Sexta-feira Santa
4 – Recesso
8 e 9 – Solicitação de ajuste de matrícula
8 e 9 – Matrícula em disciplinas de outro curso
9 – Início do semestre 2026/1
9 a 20 – Outras atividades acadêmicas (10 dias)
10 a 30 – Ajuste de matrícula pelas Coordenações
21 – Feriado – Tiradentes
22 – Início das aulas do semestre 2026/1
27 – Término do prazo de equivalências curriculares
29 e 30 – Matrícula de estudante especial

MAIO 9 aulas

aula de reposição

D	S	T	Q	Q	S	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

1 – Feriado – Dia do Trabalhador
4 –Limite resposta matrícula outro Curso,
4/5 a 4/6 – oferta de disciplinas para 2026/2
6 – Reposição aula de quinta-feira Uberlândia
9 – Prazo para retificação de notas e faltas 2025/2
19 – 25% de aulas do semestre 2026/1
27 – Prazo para trancamento parcial

JUNHO 8 aulas

D	S	T	Q	Q	S	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

4/5 a 4/6 – oferta de disciplinas para 2026/2
4 – Feriado – Corpus Christi
22 a 27 – Período colação de grau referente a 2025/2
15 – 25% de aulas do ano 2026

JULHO 8 aulas

D	S	T	Q	Q	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

12 – Prazo para solicitação de dilaçãode prazo

AGOSTO 2 aulas

D	S	T	Q	Q	S	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

8 – Término do semestre e das aulas 2026/1

Aula	Data	Conteúdo
01	23/04	Introdução à Computação Gráfica Moderna Regras da Disciplina, Histórico e Aplicações
02	28/04	Infraestrutura de Hardware
03	30/04	Infraestrutura de Software
04	05/05	Primitivas Gráficas – Linhas
05	06/05	Primitivas Gráficas – Preenchimento de Polígonos e Circunferências
06	07/05	Primitivas Gráficas – Curvas Paramétricas
07	12/05	Algoritmos de Recorte – Cohen-Sutherland
08	14/05	Algoritmos de Recorte – Ponto Médio
09	19/05	Algoritmos de Recorte – Recorte de Polígonos
10	21/05	Transformações Geométricas – 2D rígidas
11	26/05	Transformações Geométricas – 2D Afins e Morfológicas
12	28/05	Coordenadas Homogêneas
13	02/06	Matriz de Transformação Geométrica
14	09/06	Apresentação dos Trabalhos 1 e 2
15	11/06	Apresentação dos Trabalhos 1 e 2
16	16/06	***Avaliação Teórica***
17	18/06	Vista da Prova e Transformação 3D
18	23/06	Pipeline de Exibição – Câmeras Sintéticas e Passos na Visualização 3D
19	25/06	Pipeline de Exibição – Projeções Perspectivas e Paralelas
20	30/06	Pipeline de Exibição – Modelos Poliedrais e Malhas de Polígonos
21	02/07	Processamento de Sólidos – Modelagem de Sólidos
22	07/07	Processamento de Sólidos – Sólidos R-set e esquemas de representação de sólidos
23	09/07	Rendering – Algoritmo Depth-Buffer
24	14/07	Rendering – Algoritmo Z-Buffer
25	21/07	Rendering – Algoritmo Scan-Line
26	23/07	Rendering – Reflexão e Modelo de Phong
27	28/07	Rendering – Texturas
28	30/07	Modelos Globais de Iluminação: Ray Tracing, Radiosity e Modelos Híbridos
29	04/08	Apresentação dos Trabalhos 3 e 4
30	06/08	Apresentação dos Trabalhos 3 e 4

Avaliação

uma **Prova Teórica - P -** (16/06) correspondendo a 40% do conceito final;
quatro trabalhos práticos, correspondendo a 15% cada do conceito final;

- Trabalho (Módulo 1) - Mini Paint
- Trabalho (Módulo 2) - Mini Game
- Trabalho (Módulo 3) - Mundo Populado
- Trabalho (Módulo 4) - Mundo Bonito :)

A **Média do Semestre - MS -** será computada da seguinte maneira:

$$MS = Px0.4 + T1x0.15 + T2x0.15+ T3x0.15+ T4x0.15$$

Alunos que obtiverem MS no intervalo 19<MS<60 podem efetuar uma **Prova de Recuperação - PR**. A **Media Final - MF -** será computada da seguinte maneira:

$$MF = MSx0.5 + PRx0.5$$

*O aluno deve possuir frequência suficiente de acordo com as normas de graduação para ser aprovado.

Descrição dos Trabalhos Práticos

Todos os Trabalhos a seguir devem ser desenvolvidos por equipes de 2 ou 3 alunos. Não serão aceitos "grupos de 1" nem grupos de 4 ou mais alunos. As equipes de desenvolvimento serão formadas ao início do semestre e só poderão ser reorganizadas mediante justificativa plausível a critério do professor.

Trabalho 1 - Mini Paint: Implementação de um Editor Gráfico Raster com Fundamentos de Computação Gráfica"

Objetivos de aprendizado

Compreender e manipular uma matriz de pixels (framebuffer);
Implementar algoritmos de rasterização de primitivas (linhas, círculos, retângulos, polígonos);
Aplicar transformações geométricas simples (translado, rotação, escala) em objetos desenhados;
Implementar preenchimento de regiões (flood fill ou boundary fill);
Gerenciar interação do usuário (mouse/teclado) para desenho em

tempo real;
Entender o conceito de cores (RGB) e operações de blend.

Requisitos funcionais mínimos (versão básica)

O software deve permitir ao usuário:

- **Área de desenho** (canvas) de tamanho fixo (ex.: 800x600 pixels).
- **Ferramentas essenciais**:
  - Lápis (pincel de 1 pixel).
  - Borracha (pinta com a cor de fundo).
  - Linha reta (algoritmo de Bresenham ou DDA).
  - Retângulo vazado e preenchido.
  - Círculo vazado e preenchido.
  - Balde de tinta (flood fill 4 ou 8 conectado).
- **Paleta de cores** com pelo menos 8 cores pré-definidas (preto, branco, vermelho, verde, azul, amarelo, ciano, magenta).
- **Seleção de espessura** do pincel/ferramentas (3 opções: fino, médio, grosso).
- **Botão "Novo"** (limpar canvas com cor de fundo).
- **Botão "Salvar"** (exportar para um formato simples, como BMP, PPM ou PNG via biblioteca).

Requisitos técnicos (núcleo da avaliação) Sem uso de bibliotecas gráficas de alto nível

para desenho primitivo (ex.: não pode usar drawLine pronta). Você pode usar SDL2, SFML, OpenGL com glDrawPixels, ou até uma biblioteca simples como graphics.h (Turbo C++) **apenas para criar a janela e acesso ao framebuffer**.

O aluno deve implementar:

- put_pixel(x, y, cor)
- get_pixel(x, y)
- Algoritmo de linha (Bresenham).
- Algoritmo de círculo (Bresenham ou ponto médio).
- Flood fill recursivo ou com pilha/queue (não usar função pronta).
- **Tratamento de eventos do mouse** (clique, arrasto, soltar).
- **Estrutura de dados** para o canvas: matriz 2D de inteiros (cores

indexadas) ou struct RGB.

Critérios de avaliação

Critério	Peso
Funcionalidades obrigatórias completas	40%
Correta implementação dos algoritmos de rasterização (sem trapaça com funções prontas)	25%
Interface usável e responsiva (feedback visual, mouse)	15%
Organização do código (modul., comentários, clareza)	20%

Trabalho 2 - Mini Game: Jogo 2D com Transformações Gráficas e Detecção de Colisão

Objetivos de aprendizado

- Aplicar transformações geométricas (**translação, rotação, escala**) em objetos animados.
- Gerenciar uma cena dinâmica com múltiplos elementos em movimento.
- Implementar **detecção de colisão** (bounding box, círculo ou pixel perfeito).
- Utilizar **matrizes de transformação** para movimentar e rotacionar personagens/naves.
- Controlar **animação por quadros** (game loop com delta time ou passo fixo).
- Integrar eventos de teclado/mouse em tempo real.
- Aplicar **rasterização de primitivas** (opcional para projéteis/ explosões).

Requisitos funcionais mínimos

O jogo deve conter:

- **Personagem principal controlável** (ex.: nave triangular, tanque, ou nave espacial) que:
  - Move-se para esquerda/direita/cima/baixo (ou apenas horizontal na parte inferior da tela).
  - Rotaciona para mirar (se for um jogo de tiro com ângulo

variável).
Atira projéteis/pular.

Inimigos ou obstáculos (ex.: asteroides, blocos, ou inimigos simples) que:
Movem-se em trajetória linear ou ondulatória.
Aparecem em intervalos regulares (spawn).
Causam perda de vida/pontuação ao colidir com o jogador.

Sistema de pontuação:
Pontos ao destruir inimigos (com um tiro).
Exibição do placar na tela.

Condições de fim de jogo:
Vida do jogador (ex.: 3 colisões e game over).
Tela de "Game Over" e opção de reiniciar (tecla R).

Loop de jogo funcional:
Atualização de posições (lógica) e redesenho a cada frame.
Controle de taxa de quadros (ex.: 60 FPS).

Requisitos técnicos (núcleo da avaliação) Sem usar engines prontas (Unity, Godot, Unreal).

Pode usar SDL2, SFML, Pygame, OpenGL, mas:
O aluno deve implementar **manualmente** a transformação dos vértices do personagem (ex.: uma nave representada por 3 vértices) aplicando matriz de rotação e translação.
Exemplo:

$$x' = x * \cos(\theta) - y * \sin(\theta) + tx$$
$$y' = x * \sin(\theta) + y * \cos(\theta) + ty$$

Detecção de colisão implementada por ele (não usar sprite.colliderect pronta de alto nível).
Desenho de primitivas (linhas, polígonos) para representar naves e tiros – pode usar formas geométricas simples, sem sprites bitmap.
Uso de pelo menos duas transformações geométricas:
Translação (movimento da nave, inimigos, tiros).
Rotação (nave apontando para o mouse ou teclado).
Escala (opcional, ex.: inimigo cresce/encolhe).

Critérios de avaliação

Critério	Peso
Jogo jogável (sem crashes, controles funcionam, regras claras)	30%
Implementação correta de transformações geométricas (rotação/translação)	25%
Detecção de colisão funcional e correta	30%
Organização do código (game loop, update/render separados)	15%

Trabalho 3 - Passeio 3D: Visualizador Interativo de Cena com Câmera Livre

Objetivos de aprendizado

- Compreender o pipeline gráfico 3D (modelo → mundo → view → projeção → viewport).
- Implementar ou utilizar transformações **model-view-projection** (MVP).
- Criar uma **câmera controlável pelo usuário** (first-person ou órbita).
- Carregar e visualizar objetos 3D (primitivos ou modelos simples).
- Aplicar iluminação básica (luz ambiente + difusa).
- Gerenciar profundidade (teste Z / depth buffer).
- Compreender projeção perspectiva e parâmetros (FOV, near/far plane).

Requisitos funcionais mínimos

O software deve permitir:

- **Cena 3D contendo pelo menos 5 objetos** posicionados em diferentes locais do mundo, com pelo menos **dois tipos de primitivas**:
  - Cubo (ou caixa)
  - Esfera
  - Cilindro
  - Cone
  - Plano (chão)
- **Câmera controlável pelo usuário** com **pelo menos um** dos modos:
  - **First-person (FPS)** – WASD movem, mouse olha em volta.

Orbital – gira em torno de um ponto central (ex.: uma estátua).
Terceira pessoa simples – segue um personagem que se move.

Movimentação da câmera em 6DOF ou simplificada:
Avançar/recuar (W/S)
Lateral esquerda/direita (A/D)
Olhar para cima/baixo (mouse ou teclas setas)
(Opcional) Subir/descer (Q/E)

Obstáculos ou limites para não atravessar objetos (colisão simples com bounding boxes, opcional mas recomendado).

Interface simples:
Exibir coordenadas da câmera (opcional)
Tecla R para resetar posição da câmera
Tecla ESC para sair

Requisitos técnicos

Uso obrigatório de alguma API gráfica 3D:
OpenGL (moderno 3.3+) com GLFW/GLAD – recomendado.
DirectX 11 (se Windows e com suporte).
WebGL (via Three.js – mais alto nível).
Unity/Unreal apenas com restrição: o aluno deve mostrar que entende as transformações (não só usar componentes prontos sem explicação).

O aluno deve implementar explicitamente:
A matriz de projeção perspectiva (ou usar glm::perspective, mas entendendo os parâmetros).
A matriz de view (câmera) – pode ser glm::lookAt com parâmetros calculados por ele.
A transformação individual de cada objeto (posição, rotação, escala) no mundo.
Carregamento de modelos (se usar arquivos externos):
Formatos simples: .obj (sem texturas complexas, apenas vértices e faces).
Ou gerar objetos proceduralmente (cubo, esfera via latitude/longitude).

Iluminação mínima:
Uma luz direcional fixa.

Luz ambiente.
(Opcional) Material difuso para cada objeto.

Critérios de avaliação

Critério	Peso
Cena 3D carregada corretamente (objetos visíveis, profundidade funcionando)	25%
Câmera controlável (WASD + mouse/setas) com resposta suave	25%
Uso correto das matrizes de transformação (Model, View, Projection)	20%
Iluminação básica implementada (ambient + diffuse)	20%
Organização do código (classe Camera, cena, objetos)	20%

Trabalho 4 - Livre

mesmos critérios avaliados anteriormente

Bibliografia

*Sugerida na ficha da disciplina

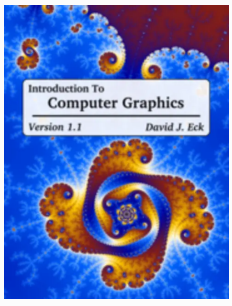
BIBLIOGRAFIA BÁSICA

1. HETEM JUNIOR, Annibal. **Computação gráfica**. Rio de Janeiro: LTC, 2006.
2. CONCI, Aura. **Computação gráfica**: teoria e prática. Rio de Janeiro: Elsevier, c2008.
3. GOMES, Jonas de Miranda. **Fundamentos da computação gráfica**. Rio de Janeiro: IMPA, 2008.

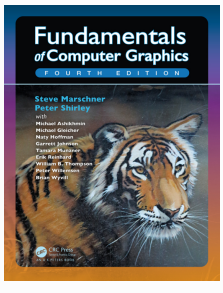
BIBLIOGRAFIA COMPLEMENTAR

1. BANON, Gerald J. F. **Bases da computação gráfica**. Rio de Janeiro: Campus, c1989.
2. GOMES, Jonas de Miranda. **Computação gráfica**: imagem. Rio de Janeiro: IMPA: SBM, 1994.
3. MAGALHÃES, Leo Pini. **Computação gráfica**: interfaces em sistemas de computação gráfica. Campinas: Ed. da UNICAMP, 1986.
4. VENETIANER, Tomas. **Desmistificando a computação gráfica**. São Paulo: McGraw-Hill, c1988.
5. PERSIANO, Ronaldo Cesar Marinho. **Introdução a modelagem de curvas e superficies em computação gráfica**.; Uberlândia: SBC: EDUFU, 1989.

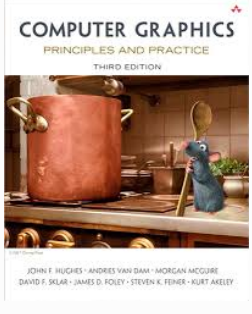
Bibliografia Sugerida (Professor)



DAVID J. ECK; HOBART AND WILLIAM SMITH. Introduction to Computer Graphics. Open Textbook Library. 2016. https://open.umn.edu/opentextbooks/textbooks/420



STEVE MARSCHNER; PETER SHIRLEY. Fundamentals of Computer Graphics. 4th ed. CRC Press. 2016.



APA: Hughes, J. F., Van Dam, A., McGuire, M., Sklar, D. F., Foley, J. D., Feiner, S. K., & Akeley, K. (2013). *Computer graphics: Principles and practice* (3rd ed.). Addison-Wesley Professional.

O que é Computação Gráfica?

- Atualmente associado a visualização iterativa, simulação e entretenimento.
- Para se responder de maneira adequada esta questão faz-se necessário entender como CG se coloca no ecosistema da computação.

- Considere:
  - Photoshop é uma aplicação de computação gráfica ou de processamento de imagens?
  - E o MS word/excel? Podemos dizer que eles utilizam computação gráfica?



O que é Computação Gráfica?

- De maneira resumida, computação gráfica trata da apresentação de informação computacional sob a forma gráfica.
- Formalmente: *"Sub-área da Ciência da Computação que consiste me métodos e técnicas usa-das para criar, armazenar e manipular modelos de objetos e suas imagens via computador"*.
- Computação gráfica é possivelmente o parcela de software que mais demanda do hardware de um sistema computacional.
  - Suporte em hardware e software dedicado.
  - GPUs, Placas gráficas, Entradas especiais no SO, e suporte da placa mãe.



O que é Computação Gráfica?

- Atualmente, computação gráfica é fortemente associada a computação gráfica tridimensional com foco no fotorealismo;



Áreas Correlatas

- Computação gráfica se correlaciona com diversas outras áreas da computação e matemática. São elas:
  - Matemática aplicada
  - Geometria computacional
  - Topologia computacional
  - Visão computacional
  - Processamento de imagens
  - Visualização de informação
  - Visualização científica
  - Álgebra linear
  - Arquitetura e organização de computadores



Relação entre PDI, CG e Visualização

- Outras áreas da computação no entanto lidam com informação sob a forma gráfica. São elas, resumidamente:
  - Processamento Digital de Imagens;
  - Computação Gráfica;
  - Visualização de Dados;
  - Geometria Computacional.





Breve Histórico

- Antes de 1950 – experimentos em saída gráfica feitos em impressoras;
- 1950 – Projeto sage, ligação de um computador a um visualizador de radar (tubo de raios catódicos);
- 1959 – Surge o termo "Computer Graphics" cunhado por Verne L. Hudson, Projeto da Boeing;
- 1962 – MIT projeto TX2
- 1970 – Surgem os primeiros monitores à varredura;
- Entre 1970 e meados de 1980 – Computadores da IBM e da DEC (computadores e minicomputadores respectivamente) disponibilizavam gradualmente mais e mais acesso a monitores gráficos
- Com a introdução do computador pessoal, a utilização de televisores e monitores se tornou lugar comum.



Hardware para CG

- A apresentação de informação sob a forma gráfica impõe requisições pesadas ao hardware do sistema computacional;
- Ao longo da evolução dos sistemas computacionais tal suporte em hardware mudou consideravelmente. Atualmente sistemas computacionais (sejam eles computadores pessoais tal como o PC ou smartphones) contam com processadores gráficos e memória de vídeo como recursos mínimos. No entanto nem sempre foi assim.
- Em um sistema computacional típico, minimanente estão disponíveis:
  - Barramento dedicado entre processador e memória gráfica;
  - Processador gráfico;
  - Dispositivo gráfico de saída (monitor, tela lcd, etc);
  - Um ou mais dispositivos de entrada de informação para iteragir com o sistema gráfico (tal como mouse, touch pad, touch screen, etc)



Dispositivos de Saída

- Forma primária para apresentação da informação gráfica;
- Evoluiram drasticamente ao longo dos últimos 50 anos;
  - impressoras;
  - Monitores da varredura;
  - Monitores LCD/ LED;
  - Etc.
- Ao longo da história dois modelos de apresentação gráfica estiveram em voga. São eles:
  - Modelo Vetorial;
  - Modelo Raster.



Architetura de Visualização Vetorial



Imlac PDS-1

- Derivada originalmente dos monitores de radares;
- O tubo de raios catódicos ativa os depósitos de fósforo apenas nas posições definidas por uma primitiva vetorial.
- Por exemplo para se desenham uma linha dois pontos são especificados. O tubo de raios catódicos é então movimentado no caminho entre estes pontos excitando os depositos de fósforo.
- Otimo para primitivas gráficas, geometria, mas problemático no quesito fotorealismo.



Arquitetura Raster

- O modelo raster ou de varredura demanda muito mais recursos de hardware que o modelo vetorial. Devido a tal fato ele foi lentamente aceito.
- No entanto hoje é o modelo de fato, disponível virtualmente em todos os dispositivos gráficos.
- Seu funcionamento é simples. Basicamente, cada elemento gráfico (pixel) do dispositivo de saída é mapeado em um banco de memória. Todos os valores de cor de cada pixel são atualizados a cada varredura. Monitores geralmente atualizam todos os pixels da tela a cada 1/60 segundos.



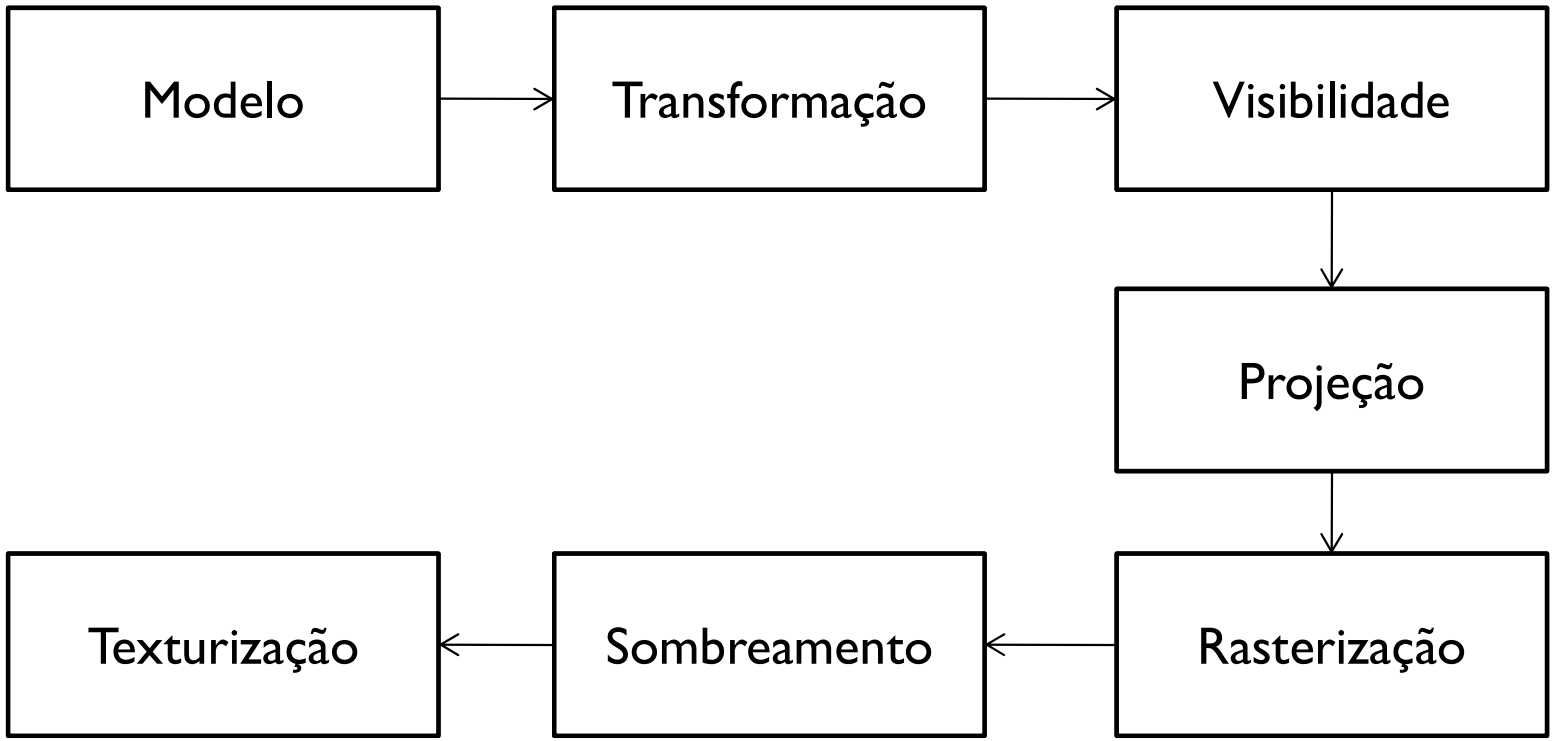
Coordenadas do Mundo vs da Janela vs do Dispositivo



- Uma cena é geralmente descrita por meio de um modelo, que nada mais é que um conjunto de primitivas gráficas associadas a modificadores de cor/textura/sombreamento, etc. O modelo é gerado pelo sistema gráfico de maneira independente do sistema de visualização. Sendo assim, há liberdade para se utilizar o sistema de coordenadas que melhor aprouver ao modelo.
- No entanto ele deve ser apresentado em um sistema gráfico. Sendo assim, as coordenadas do modelo devem ser convertidas para as coordenadas no dispositivo, ou seja, display gráfico.
- O problema de conversão de coordenadas é complicado ainda pela possibilidade de que nem todo o modelo do mundo seja visualizado por vez. Neste caso, apenas uma porção, ou janela, do modelo pode ser selecionada, requerendo uma conversão de coordenadas intermediária.



Pipeline de Visualização



- Todo sistema gráfico possui em maior ou menor detalhamento os sete subsistemas apresentados acima.



Modelo

- Descrição matemática de:
  - Objetos
  - Plano de fundo
  - Fontes de luz
  - Posicionamento da cena (câmera)
  - Parâmetros de sombreamento
  - Etc
- Em modelos tridimensionais:
  - Listas de vértices/arestas
  - Listas de faces
- Essencialmente, o Modelo é uma descrição da cena. Hhá diversas formas de descrever um modelo. Pode-se utilizar uma descrição primitiva usando listas de vértices, faces e arestas, pode-se utilizar primitivas gráficas como cubos, esferas, etc.



Transformação

- A construção de uma cena requer que os diversos objetos que a compõem sejam reposicionados, geralmente em relação ao ponto de visão (câmera). Tal reposicionamento é feito por meio de transformações geométricas:
  - Rígidas: translação, rotação e espelhamento
  - Afins: Zoom e alongamento (Sheer)
  - Projetivas
  - Elásticas



Visibilidade

- Dependendo do posicionamento da câmera, pode ser que nem toda a cena seja visível. Consequentemente tais objetos localizados fora do campo de visão não precisam ser processados.
- Mesmo que um objeto esteja posicionado dentro da janela de visualização mas no entanto parcialmente ou totalmente ocluso por outro objeto, este deve ser considerado como quesito de visibilidade o que impacta o desempenho do pipeline gráfico.



Projeção

- Simula matematicamente o posicionamento da câmera (olhos) e a produção de uma projeção bidimensional da cena.



Rasterização

- É a parte do pipeline gráfico responsável por transformar a descrição contínua do modelo matemático da cena em uma lista discreta de posições em tela. Geralmente, anti-aliasing também é tratado nesta etapa do pipeline gráfico.



Sombreamento

- (Shading) é a parte do pipeline gráfico responsável por simular o efeito da iluminação na cena.

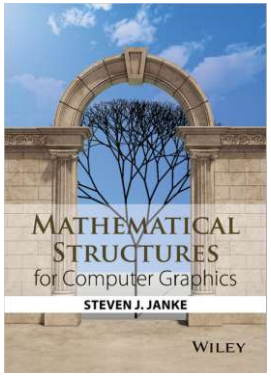


Texturização

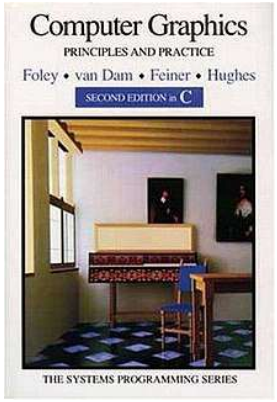
- Parte do pipeline gráfico responsável por descrever a superfície de objetos. Geralmente refere-se ao mapeamento dos pixels de uma imagem 2D (textura) para as faces do modelo 3D.



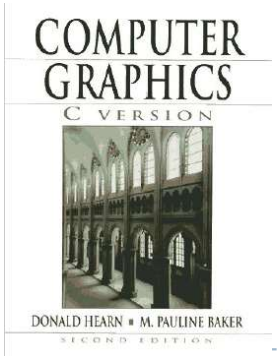
Bibliografia



[JAN2015] Janke, S. J. “Mathematical structures for computer graphics”. 2015. John Wiley & Sons, Inc. ISBN: 978-1-118-71219-1. 411p.



[FOL1994] Foley, J.D. *et al. “Computer graphics: principles and practice”. Addison-Wesley. 1994.*



[Hea1997] Hearn, D. & Baker, M. P. “Computer Graphics, C Version”. 1997. Prentice Hall, 2ª Ed. ISBN: 978-0-13530-9247. 652p.



Referências

- **When a Bit Became a Pixel: The History of Computer Graphics**
  - https://www.youtube.com/watch?v=iw1o4ozvjEU&list=PLD3uaVhJdwqWW6u5EpzGqOfeppIPhIu2O&index=2&t=0s
- **Uma Breve História dos Gráficos**
  - https://www.youtube.com/watch?v=QyjyWUrHsFc&index=3&t=0s&list=PLD3uaVhJdwqWW6u5EpzGqOfeppIPhIu2O
- **Learn OpenGL**
  - https://learnopengl.com/
  - https://learnopengl.com/book/learnopengl_book.pdf

